```
end
```

This migration will create a `user_id` column and appropriate index.

There is also a generator which will produce join tables if `JoinTable` is part of the name:

```
$ bin/rails g migration CreateJoinTableCustomerProduct customer
product
```

will produce the following migration:

```
class CreateJoinTableCustomerProduct < ActiveRecord::Migration
  def change
    create_join_table :customers, :products do |t|
      # t.index [:customer_id, :product_id]
      # t.index [:product_id, :customer_id]
    end
  end
end
```

2.2 Model Generators

The model and scaffold generators will create migrations appropriate for adding a new model. This migration will already contain instructions for creating the relevant table. If you tell Rails what columns you want, then statements for adding these columns will also be created. For example, running:

```
$ bin/rails generate model Product name:string description:text
```

will create a migration that looks like this

```
class CreateProducts < ActiveRecord::Migration
  def change
    create_table :products do |t|
      t.string :name
      t.text :description

      t.timestamps null: false
    end
  end
end
```

You can append as many column name/type pairs as you want.

2.3 Passing Modifiers

Some commonly used type modifiers can be passed directly on the command line. They are enclosed by curly braces and follow the field type:

For instance, running:

```
$ bin/rails generate migration AddDetailsToProducts
'price:decimal{5,2}' supplier:references{polymorphic}
```

will produce a migration that looks like this

```
class AddDetailsToProducts < ActiveRecord::Migration
  def change
    add_column :products, :price, :decimal, precision: 5, scale: 2
    add_reference :products, :supplier, polymorphic: true, index: true
  end
end
```

Have a look at the generators help output for further details.